



Universidade do Minho
Escola de Engenharia

Laboratórios de Informática

Licenciatura em Engenharia Informática

Grupo 64

2022/2023

1ª fase

Alexandra Santos (A94523)
Guilherme Oliveira (A100592)
Gustavo Castro (A100482)

Braga, de Novembro de 2022

1.- Introdução	3
2.- Estruturas de Dados	3
2.1.- Estruturas de Armazenamento	3
2.1.1.- Estrutura de User	3
2.1.2.- Estrutura de Driver	4
2.1.3.- Estrutura de Ride	4
2.2.- Estruturas de Procura	5
2.2.1.- Filtro de Cidades	5
2.2.2.- Filtro de Datas	5
3.- Modularidade	6
3.1.- Módulos de estrutura de dados	6
3.2.- Módulos de leitura e resultado	6
3.3.- Módulos de funcionalidade	6
4.- Encapsulamento	7
4.1.- Encapsulamento nos módulos de estruturas	7
4.2.- Encapsulamento nos módulos de IO	7
4.3.- Encapsulamento nos módulos de funcionalidade	7
5.- Abstração de Dados	8
5.1.- Hashmap	8
5.2.- HashmapNodes	8
5.3.- Método de Chaining	8
6.- Estratégias das Queries	9
6.1.- Query 4	9
6.2.- Query 5	9
6.3.- Query 6	9
7.- Desempenho do programa	10
8.- Conclusão	10

1.- Introdução

Este documento apresenta o projeto de Laboratórios de Informática 3 (LI3), do curso de Engenharia Informática da Universidade do Minho. O projeto baseia-se na criação de uma aplicação capaz de armazenar informações, fornecidas a partir de ficheiros .csv, numa estrutura de dados de forma a que essa informação possa ser usada na realização de **Queries**. Pretende-se que o programa recolha os dados da estrutura e realize os cálculos necessários à conclusão dos comandos executados.

2.- Estruturas de Dados

Para o melhor funcionamento deste programa foi optado por dividir os vários dados vindos dos documentos .csv em dois tipos de estruturas de dados: as principais (de armazenamento) e as secundárias (de procura), que serão distribuídas por um terceiro tipo de estrutura que será referida na secção de Abstração de Dados.

2.1.- Estruturas de Armazenamento

As estruturas de dados principais guardam toda a informação relativa aos dados principais (Utilizadores, Condutores e Viagens) de forma a que seja facilmente acessível quando for necessária.

Desta forma, é recorrida a criação de três novos tipos de variáveis para: armazenar utilizadores (User); armazenar condutores (Drivers); e armazenar viagens (Ride).

2.1.1.- Estrutura de User

A estrutura de User armazena a informação relativa a um Utilizador e apresenta as seguintes propriedades:

- **Username (char[50]):**
Servirá como um tipo de ID para o Utilizador, já que o Username é único de cada utilizador.
- **Name (char[50]):**
É o nome do Utilizador em questão.
- **Gender (char):**
É o género do Utilizador, que será representado apenas pela primeira letra (M - Masculino ou F - Feminino).
- **Birth_date (short[3]):**
Representa a data de nascimento de um Utilizador. Esta será representada por um array de short ints, onde o primeiro elemento é o dia, o segundo elemento é o mês e o terceiro o ano.
- **Account_creation (short[3]):**
Armazena a data de criação da conta de um Utilizador.
- **Pay_method (char[50]):**
Define o método de pagamento utilizado por um Utilizador.
- **Account_status (char):**

É a propriedade que define se um Utilizador está ativo ou inativo, usando apenas a primeira letra (caracter) do estado para o fazer.

2.1.2.- Estrutura de Driver

A estrutura de Driver guarda a informação referente a um Condutor e é definida pelas seguintes características:

- **ID (int):**
Tal como o nome indica, será usado como ID (numérico) único de cada Condutor.
- **Name (char[50]):**
É o nome do Condutor a ser tratado.
- **Birth_day (short[3]):**
Guarda a data de aniversário de um Condutor.
- **Gender (char):**
Armazena o género do Condutor, usando para isso a primeira letra do respetivo.
- **Car_class (char):**
Define o tipo de carro que um Condutor conduz, usando a seguinte regra: (0 - Basic, 1 - Green, 2 - Premium).
- **License_plate (char[10]):**
Representa a matrícula de um veículo de um Condutor.
- **City (char[30]):**
Define a cidade onde um Condutor faz as suas viagens.
- **Account_creation (short[3]):**
Armazena a data de criação da conta de um Condutor.
- **Account_status (char):**
Guarda o estado da conta, usando apenas a primeira letra do estado para tal.

2.1.3.- Estrutura de Ride

A estrutura de Ride define a informação associada a um Condutor e é definida pelas seguintes propriedades:

- **ID (int):**
À semelhança da estrutura definida anteriormente, esta propriedade também define o ID (numérico) único de cada Viagem.
- **Date (short[3]):**
É a data da ocorrência de uma Viagem.
- **Driver (int):**
Guarda o ID do Condutor da Viagem em questão.
- **User (char[50]):**
Guarda o Username do Utilizador da Viagem em questão.
- **City (char[50]):**
Armazena o nome da cidade onde a Viagem foi realizada.
- **Distance (short):**
Define a distância total percorrida durante a Viagem.
- **Score_user (short):**

- É definido pela pontuação dada ao Utilizador da respetiva Viagem.
- **Score_driver (short):**
É definido pela pontuação dada ao Condutor da respetiva Viagem.
- **Tip (float):**
Define a gorjeta dada pelo Utilizador de uma Viagem.
- **Comment (char[100]):**
Armazena o comentário deixado pelo Utilizador relativo a uma Viagem.

2.2.- Estruturas de Procura

As estruturas de dados secundárias armazenam toda a informação que possa ser útil para fins de filtragem de informação, de forma a que a informação seja facilmente acessível mesmo que não se possua o ID único de cada elemento das várias estruturas descritas anteriormente.

Atualmente, foram definidas apenas duas estruturas (filtros) deste tipo: filtros de cidades (City); e filtros de datas (Date).

2.2.1.- Filtro de Cidades

O filtro de cidade é definido pela informação relativa à cidade e às suas respetivas ocorrências nas várias estruturas de armazenamento.

A sua estrutura apresenta as seguintes definições:

- **City (char[50]):**
Armazena a cidade que poderá ser usada como filtro.
- **Key (int):**
Armazena o ID (numérico, já que só Condutores e Viagens apresentam propriedades relativas a cidades) de uma ocorrência da cidade.
- **Type (char):**
Define o tipo de ID armazenado na propriedade anterior (r - Viagem, d - Condutor).

2.2.2.- Filtro de Datas

O filtro de data armazena a informação correspondente a uma data e às suas respetivas ocorrências nas várias estruturas de armazenamento.

A sua estrutura apresenta as seguintes propriedades:

- **Data (short[3]):**
Guarda uma data que poderá ser usada como filtro.
- **KeyRef (void *):**
Armazena o pointer do ID (visto que pode ter origem de qualquer estrutura de dados) de uma ocorrência da data.
- **Type (char):**
Define o tipo de ID armazenado na propriedade anterior (u - Utilizador, r - Viagem, d - Condutor).

Estas estruturas serão depois armazenadas numa lista ligada (referida na secção de Abstração de Dados), que permitirá o armazenamento de mais do que uma ocorrência por filtro.

3.- Modularidade

Tendo em consideração a melhor organização e desempenho do programa, as várias funcionalidades e estruturas foram divididas por diversos módulos.

Estes módulos foram divididos por três tipos:

- Módulos de estrutura de dados (structs);
- Módulos de leitura e resultado (io);
- Módulos de funcionalidade;

3.1.- Módulos de estrutura de dados

Este tipo de módulos tem como objetivo definir as estruturas de dados usadas ao longo do programa, assim como funções úteis à sua utilização (criação, destruição ou busca das estruturas).

Usando esta tipologia, foram criados os seguintes módulos:

- User (Armazenamento de utilizadores);
- Driver (Armazenamento de condutores);
- Ride (Armazenamento de viagens);
- Date (Filtro de datas);
- City (Filtro de cidades);
- Global (Armazenamento global);
- Hashmap (Abstração de dados);

3.2.- Módulos de leitura e resultado

Nesta tipologia, o objetivo é definir os métodos de leitura do input do utilizador e de criação de resultados dependentes do que foi lido no mesmo input.

Assim sendo, recorreu-se à utilização dos dois seguintes módulos:

- In (Leitura do input (.csv e lista de comandos));
- Read (Interpretação da informação e criação de resultados dependentes da mesma);

3.3.- Módulos de funcionalidade

Nos módulos de funcionalidade, o objetivo é tratar a informação recebida dos dois módulos anteriores, e devolver resultados baseados no que é recebido.

Desta forma, são definidos dois novos módulos:

- Utils (Utilidades, conversões, etc...);
- Queries (Tratamento estatístico da informação);

4.- Encapsulamento

De forma a encapsular o programa, às várias funcionalidades de cada módulo são atribuídas permissões. Estas permissões irão definir quais módulos têm acesso a quais funcionalidades, permitindo, desta forma, uma maior segurança no programa, sendo que algumas funcionalidades podem até apenas poder ser acedidas pelo próprio módulo (no caso de funções auxiliares a funções principais).

4.1.- Encapsulamento nos módulos de estruturas

Na generalidade, todos os módulos de estruturas de dados são independentes, recorrendo excepcionalmente a funções do módulo **Utils** para simplificar os seus processos.

Tirando esse uso, recorrem apenas às suas próprias funções.

4.2.- Encapsulamento nos módulos de IO

Em contraste à categoria anterior, os dois módulos de IO divergem bastante em termos de permissões de acesso aos restantes módulos.

O módulo **IN** tem acesso ao módulo **Read**, de forma a chamar as funções de interpretação presentes no mesmo; e ao módulo **Global**, conseguindo, assim, manter a mesma Global ao longo da interpretação da informação do input do utilizador.

Já o módulo **Read** tem acesso aos módulos de estruturas, para poder criar todas as estruturas necessárias consoante a interpretação do input do utilizador; ao módulo **Utils**, visto que são necessárias funções deste módulo para conversões, criação e comparações de dados; e ao módulo **Queries**, onde apenas tem acesso às funções principais das queries, que serão usadas para executar as queries escolhidas pelo utilizador.

4.3.- Encapsulamento nos módulos de funcionalidade

Os módulos de funcionalidade, tal como os da categoria anterior, divergem, também, em permissões, sendo que o módulo **Utils** apenas tem acesso a ele próprio.

Por oposição, o módulo **Queries** tem acesso aos módulos de estruturas, tal que consegue ler e tratar da informação; e ao módulo **Utils**, usando as comparações, conversões e criação de dados para auxiliar no tratamento de dados.

Por último, o “módulo” **Main** do programa, que o executa, apenas tem acesso ao módulo **IN**, usando a sua função para iniciar a leitura e interpretação do input; e ao módulo **Global**, conseguindo, assim, criar a estrutura global que sustentará a execução inteira do programa.

5.- Abstração de Dados

A fim de obter a melhor organização e desempenho possível, foi preferido o uso de um **Hashmap** para armazenamento dos dados e filtros discutidos anteriormente.

Através do **Hashmap** será possível organizar os dados de forma eficiente, visto que cada elemento deste tem um identificador próprio (ou cada lista, no caso dos filtros), sendo assim possível rapidamente devolver o elemento ou lista inteira do **Hashmap** sabendo apenas o seu respetivo identificador.

5.1.- Hashmap

Mentalizando toda a informação conhecida acerca do conteúdo que deve ser tratado, o **Hashmap** opta por reservar 100000 posições à partida, que à medida que este for sendo enchido com elementos, vão ser usadas para armazenar **HashmapNodes**.

Pode ser, então, definido que o Hashmap será um array de 100000 **Nodes**.

5.2.- HashmapNodes

Cada **HashmapNode** pode ser visto como um espaço de uma lista que armazenará a informação necessária ao elemento que este contém e à sua respetiva identificação, tal como, a ligação ao próximo espaço da lista.

Esta estrutura de lista permitirá o funcionamento do Hashmap através do *Chaining* para o tratamento de colisões, tal como o uso desta para filtragem de informação, usando a lista como forma de armazenar todas as ocorrências de um filtro com o mesmo identificador (por exemplo cidades e datas).

A estrutura de uma **HashmapNode** pode, então, ser definida por:

- **Key (void*):**
Guarda o apontador da chave do elemento.
- **Data (void *):**
Armazena o apontador da informação referente ao elemento armazenado no **Node**.
- **Next (struct _HASHMAP_NODE_):**
Armazena o apontador do próximo **Node** na lista ligada.

5.3.- Método de Chaining

O **Hashmap** usa três funções de hash: para identificadores de string; para identificadores numéricos inteiros; e para identificadores do tipo **Date (short[3])**. Estas têm como utilidade adquirir a posição onde está ou vai ser guardado o elemento no Hashmap a partir do seu respetivo identificador.

Todas as funções procuram atribuir uma posição a cada chave através de cálculos matemáticos. No entanto, é possível ser atribuída a mesma posição a dois elementos distintos (colisão). Para resolver este conflito, é usada a estrutura de lista dos **Nodes**. O espaço que já se encontrava na posição passa a ser o segundo elemento na lista ligada de **Nodes** dessa posição e o espaço novo passa a ser o

primeiro elemento, criando um apontador para este segundo, de tal forma a que este não seja perdido, e assim sucessivamente no caso de futuras colisões.

6.- Estratégias das Queries

6.1.- Query 4

Esta query tem como objetivo calcular o preço médio das viagens, sem considerar as gorjetas, de uma determinada cidade, que é representada por <city>.

Assim sendo, a query vai escolher a lista da cidade que pretendemos calcular a média. Após ter escolhido esta, vai ser chamada à função `preco_medio`, que irá percorrer a lista, elemento por elemento. Por cada elemento que for percorrido, vai-se buscar a ocorrência desse elemento ao **Hashmap** das viagens, e de seguida, através da viagem obtida, vai-se buscar o condutor dessa viagem. Após ter estes valores, a função vai calcular o preço médio através da classe de veículo e da distância percorrida.

Por fim, faz a média de todos os preços calculados e devolve o resultado.

6.2.- Query 5

Esta query tem como objetivo calcular o preço médio das viagens, sem considerar gorjetas, num dado intervalo de tempo, sendo esse intervalo representado por <dataA> e <dataB>.

De forma a atingir este objetivo é criada uma função que recebe as duas datas e o tipo de **Date** que se pretende encontrar. Esta percorre o **Hashmap** de **Dates** e procura desde a dataA à dataB todos os elementos desse **Hashmap** que são do tipo pretendido. Sempre que encontra uma **Date** deste tipo, a função armazena esse **HashmapNode** numa lista ligada de **HashmapNode**, que no final da função é retornada.

Posteriormente, a lista de **Dates** retornada é fornecida à função `preco_medio()` que foi criada com o objetivo de calcular o preço de cada viagem somando os valores obtidos. Por fim, esta função divide o somatório pelo número de viagens da lista ligada, obtendo assim o preço médio das viagens entre as datas dataA e dataB.

O resultado é retornado terminando assim a query.

6.3.- Query 6

Esta query objetiva calcular a distância média percorrida numa cidade dentro de um dado intervalo de tempo.

Recebe, então, como input os respetivos filtros mencionados acima, e usa-os para obter as suas medições.

Através da cidade escolhida para filtrar, esta irá encontrar a lista de procura referente à mesma cidade e irá percorrê-la de forma a passar por todos os seus elementos.

Ao fazer este percurso, irá verificar se a data de cada elemento está dentro do intervalo de tempo escolhido, e, no caso de a condição ser respeitada, considerará a distância percorrida deste elemento para a média da distância total percorrida.

Percorridos então todos elementos e estando a média da distância calculada, esta retornará o valor desejado.

7.- Desempenho do programa

Considerando toda a estrutura e funcionamento descritos até agora, foram feitas medições que avaliam o desempenho do programa.

Assumindo a base de dados fornecida para esta primeira fase do programa, os resultados obtidos foram:

- **Gastos (em média) na memória da execução total do programa:**
Aproximadamente 350 MBytes.
- **Tempo (em média) gasto na leitura e interpretação da informação:**
Aproximadamente 1 segundo.
- **Tempo (em média) gasto na execução por query:**
Aproximadamente 150 milissegundos.

Através destes resultados, é possível, então, concluir que o programa tem um bom desempenho.

8.- Conclusão

Nesta primeira fase do projeto conseguimos desenvolver o nosso conhecimento sobre a modularização e encapsulamento. Deste modo, foi possível compreender a estratégia e a implementação demonstrável na leitura e interpretação dos dados de entrada e na utilização do modo de operação batch. Outro objetivo atingido, foi a criação de 3 das queries com base no catálogo de dados dos três ficheiros de entrada.